

Performance Analysis — RTWave_SHM Ray Tracing

Executive Summary

The simulation traces ~4000 initial rays (2001 directions $\times$ 2 modes) through a partitioned plate with 24 segments. Each boundary hit spawns child rays, leading to exponential growth. The dominant bottleneck is the **HDF5 serialize/deserialize cycle on every trace step**, followed by **redundant FFT/interpolation work** and **unchecked ray proliferation**.

The improvements below are ordered by estimated impact.

BOTTLENECK 1 — HDF5 Save/Load on Every Ray (CRITICAL)

Where: `map_2d.py:calc_t` $\rightarrow$ `load_ray` / `save_ray` (called per ray, per trace step)

What happens: Every ray is serialized to HDF5 via `np.column_stack` + `np.concatenate` on save, and reconstructed with full `Ray.__init__` (including `fft_speed` computation) on load. This is the innermost loop — every single ray trace step pays this cost.

```
python

# calc_t hot loop:
for i in range(len(self.rays_h)):
    ray = self.get_ray(self.rays_h[i]) # load_ray: HDF5 read + Ray.__init__
    rays_r = self.trace_ray(ray, t)    # trace  $\rightarrow$  intersect  $\rightarrow$  save_ray
    self.rays_h += rays_r
```

Cost per ray:

- `save_ray`: `np.column_stack` + `np.concatenate` (allocates ~500-col matrix), HDF5 `create_dataset` or `resize` + write
- `load_ray`: HDF5 read, `np.real()` slicing, full `Ray.__init__` with `fft_speed` loop (500 wavespeed lookups), conversion to Python `list`

Recommendation: Keep rays in memory using a dictionary `{hash: Ray}`. Only write to HDF5 at the end or periodically for checkpointing. The current design was likely motivated by memory concerns, but each ray object is small (~few KB). Even 100k rays $\approx$ hundreds of MB, which is manageable.

```
python
```

```
# Proposed: in-memory ray store
self.ray_cache = {}

def get_ray(self, ray_h):
    if ray_h in self.ray_cache:
        return self.ray_cache[ray_h]
    return load_ray(ray_h, self.h5file, self) # fallback

def save_ray(self, ray):
    self.ray_cache[ray.__hash__()] = ray

def flush_to_h5(self): # call at end or periodically
    for ray in self.ray_cache.values():
        save_ray(ray, self.h5file)
```

BOTTLENECK 2 — Ray.init Recomputes fft_speed Every Time (HIGH)

Where: `Ray.py:70`

```
python

self.fft_speed = np.array([self.medium.v_ray(self, fi=fi) for fi in self.fft_freq])
```

This calls `medium.v_ray` → `wavespeed_composite.S0/A0` (2D interpolation) **500 times** in a Python loop, for every new `Ray` object. When `load_ray` reconstructs a ray, it calls `Ray.__init__` again and throws away the result (the loaded data overwrites it), so these 500 interpolation calls are completely wasted on load.

Recommendation:

- **For load_ray:** Use a lightweight factory or `__new__` that skips `fft_speed` computation. Or compute `fft_speed` lazily.
- **For new rays:** Vectorize the velocity lookup. Precompute `fft_speed` once per (medium, kind, direction) combination and cache it, since many rays in the same medium with similar directions share identical dispersion curves.

```
python
```

```
# Vectorized version (replace the loop):
f_d_arr = self.fft_freq / 1e6 * medium.th
theta = np.arctan2(direction[1], direction[0]) + medium.theta
theta = theta % np.pi
# Batch interpolation instead of 500 individual calls
self.fft_speed = np.array([
    getattr(medium.ws, kind)((fd, theta)) * 1e3 for fd in f_d_arr
])

# Even better: cache per (medium_hash, kind, theta_quantized)
```

BOTTLENECK 3 — Exponential Ray Proliferation (HIGH)

Where: `ray_utils.py:ray_refl` and `ray_utils.py:ray_refr`

Each boundary intersection spawns:

- 1 mode-converted reflected ray (always)
- If 2 mediums on segment: 1 refracted + 1 mode-converted refracted

So a single hit on a damage wall creates **3 new rays** that each immediately call `.trace(t)`, which can hit another boundary and spawn more. With 24 segments (many of them invisible walls with `ratio_rfl=0`), this produces an enormous tree.

Key observation: Invisible walls (`ratio_rfl=0`, `ratio_mode=1`) still spawn a mode-converted ray with $a = a_i * 0.0 * (1 - 1.0) * (1 - 0.0) = 0.0$ amplitude. These zero-amplitude rays are created, traced, saved, and only killed later by the `a_tol` check. Similarly, boundary walls with `ratio_mode=1` produce zero-amplitude mode-converted rays.

Recommendation:

- **Skip spawning rays with zero amplitude:**

```
python

# In ray_refl:
mc_amplitude = a_i * ratio * (1 - ratio_mode) * (1 - bl)
if mc_amplitude > a_tol: # only create if it will survive
    ray_mc = mode_change(ray, mc_amplitude)
    irays.append(ray_mc.__hash__())
    irays.extend(ray_mc.trace(t, map))
```

- **Aggressive amplitude culling:** Raise `a_tol` or add a relative threshold. Many rays after 2-3 reflections have negligible amplitude but still get fully traced.

- **Limit max bounces:** Add a generation counter and cap at a reasonable depth.
-

BOTTLENECK 4 — Ray.hash() Is Expensive and Called Repeatedly (MEDIUM-HIGH)

Where: `Ray.py:359-363`

```
python

def __hash__(self):
    return hash(hash(self.kind) + hash(tuple(self.d[0])) +
               hash(tuple(self.trace_points[0])) + hash(self.int_times[0]) + hash(tuple(self.freq[0])))
```

This creates **tuples from numpy arrays** (500-element `freq[0]!`) and hashes them. It's called on:

- `ray_refl` (multiple times)
- `ray_refr` (multiple times)
- `save_ray`
- `sensor.intersect`
- `set_init_beam`

Recommendation: Compute the hash once at creation and cache it:

```
python

def __init__(self, ...):
    ...
    self._hash = hash(hash(self.kind) + hash(tuple(self.d[0])) +
                     hash(tuple(self.trace_points[0])) + hash(self.int_times[0]) +
                     hash(tuple(self.freq[0])))

def __hash__(self):
    return self._hash
```

Note: `__slots__` is already defined, so you need to add `'_hash'` to it.

BOTTLENECK 5 — Signal Reconstruction: irfft per Integration Point (MEDIUM)

Where: `Sensors.py: _signal_on_ray` → `Ray.signal_at_x`

For each sensor, for each intersecting ray, the code integrates across the sensor aperture with `d_x=0.1` mm steps. At each step it calls `signal_at_x(x)` which:

1. Searches for the correct segment (`next(i for i, v in ...)`)
2. Calls `calc_ray(i=i, x=x)` → `fshift_dispersion` (complex exponential of 500 elements)

3. Calls `irfft(f_i, n=10000)` → 10,000-point FFT

For a sensor of radius 4mm (diameter 8mm), that's ~80 integration points × (fshift + irfft) per ray.

Recommendation:

- **Reduce integration points:** Use `d_x=0.5` or `d_x=1.0` for the sensor integration, or use a single midpoint evaluation.
- **Batch the irfft:** Compute all `f_i` values first into a matrix, then call `irfft` once on the batch.
- **Precompute the `fft_freq * t_d` array:** The phase shift in `fshift_dispersion` recomputes $2\pi * \text{fft_freq} * \frac{x}{\text{fft_speed}}$ for every `x`, but `fft_freq / fft_speed` is constant.

```
python

# Precompute once per ray:
phase_rate = 2 * np.pi * ray.fft_freq / ray.fft_speed # constant

# Then fshift becomes:
f_d = np.exp(-1j * phase_rate * x) * f
```

BOTTLENECK 6 — `medium.v_ray` Overhead per Call (MEDIUM)

Where: `objects_2d.py:73-86`

```
python

def v_ray(self, ray, i=-1, fi=None):
    if fi is None:
        fi = ray.fft_freq[np.argmax(np.abs(ray.freq[i]))]
    f_d = fi/1.E+6 * self.th
    theta = np.arctan2(ray.d[i][1], ray.d[i][0]) + self.theta
    theta = theta % np.pi
    return getattr(self.ws, ray.kind)((f_d, theta)) * 1.E3
```

Issues:

- `np.argmax(np.abs(ray.freq[i]))` computes abs of a 500-element complex array every time
- `getattr(self.ws, ray.kind)` does a string lookup each call
- `np.arctan2` + modulo for a 2-element vector

Recommendation:

- Cache the dominant frequency index (it's the same for all lookups on the same ray segment)
- Replace `getattr` with direct attribute access based on kind:

```
python
```

```
_ws_lookup = {'S0': self.ws.S0, 'A0': self.ws.A0}  
# Then use _ws_lookup[ray.kind]
```

BOTTLENECK 7 — `gc.collect()` in Main Loop (LOW-MEDIUM)

Where: `02_Run-Single-Damage.py:80`

`gc.collect()` is called after every source. But inside `Map2D.trace_ray`, each call triggers `gc.collect()` indirectly through object churn. The explicit GC call in the main loop is minor, but the GC pressure from creating/destroying thousands of temporary numpy arrays in the hot path is not.

Recommendation: Disable GC during the tracing phase:

```
python
```

```
import gc  
gc.disable()  
m.calc_t()  
gc.enable()  
gc.collect()
```

BOTTLENECK 8 — `Segment.intersect` Checks All Segments (LOW-MEDIUM)

Where: `Ray.trace` → loops over `self.medium.objs`

Each ray checks intersection against every segment in its medium. With 4 segments per medium this is cheap, but after reflections, child rays inherit the same medium and check the same segments. More importantly, invisible walls (zero reflection) still trigger the full intersection test.

Recommendation:

- Pre-filter: skip segments that are known to be behind the ray (dot product check with segment midpoint)
 - Spatial indexing is overkill for 4 segments per medium, but helpful if geometries grow
-

BOTTLENECK 9 — Redundant `fft_speed` on Mode Change (LOW)

Where: `ray_utils.py:mode_change` → `Ray.__init__`

`mode_change` creates a new Ray with a different kind (S0↔A0). This triggers full `Ray.__init__`, including the 500-call `fft_speed` loop. Since mode-converted rays are spawned at every reflection, this multiplies the cost of Bottleneck 2.

Summary — Priority Implementation Order

Priority	Bottleneck	Est. Speedup	Effort
1	In-memory ray cache (skip HDF5 per step)	5–20×	Medium
2	Skip zero-amplitude ray spawning	2–5×	Low
3	Cache Ray.hash()	1.5–3×	Low
4	Lazy/skip fft_speed on load_ray	1.5–3×	Low
5	Vectorize fft_speed in Ray.init	1.3–2×	Low
6	Batch irfft in signal reconstruction	1.5–2×	Medium
7	Cache dominant freq in v_ray	1.2–1.5×	Low
8	Disable GC during tracing	1.1–1.3×	Trivial
9	Reduce sensor integration points	1.1–1.5×	Trivial

Estimated combined effect: Implementing priorities 1–5 should yield roughly **10–30× speedup** depending on the specific simulation parameters (number of bounces, ray count, etc.).